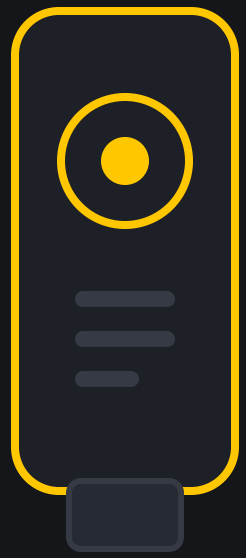
Hardware-backed KSM clients YubiKey support in the Java SDK



The client private key and the app key stay on the YubiKey.
A copied config file is useless on its own.

Keeper Secrets Manager · Java SDK · first iteration

THE PROBLEM

The KSM config file is the credential

```
// config.json today
{
  "hostname": "keepersecurity.com",
  "clientId": "Gx3h…",
  "privateKey": "MIGHAgEAMBMGByqGSM49…",
  "appKey": "q9ZtR2…",
  "appOwnerPublicKey": "BGn0…"
}
```

- **privateKey** signs every request, so whoever holds it *is* the client.
- **appKey** decrypts every record and folder key the application can reach.
- Both are plain base64 in a JSON file on disk, in CI variables, in backups, in container images.
- Copy the file and you can impersonate the client from anywhere, until someone notices and revokes the device.

GOALS

What this adds

Key custody

- The client private key is generated **on the YubiKey** and never leaves it
- Every KSM request is signed **by the YubiKey**
- The **app key** is stored only wrapped by the YubiKey
- Keys are **named after the KSM application** in YubiKey tools

Fit

- **No server changes**: same protocol, same signatures
- No native libraries and no Yubico tools needed
- One YubiKey serves **many KSM configs**
- The config says **which YubiKey and slot** holds its key

BACKGROUND

How a KSM client authenticates

1 · TOKEN
One-time token
`REGION:clientKey`. Its HMAC gives the `clientId`.

›

2 · KEY PAIR
Client P-256 key
Generated by the client. **Now on the YubiKey.**

›

3 · BINDING
First get_secret
Sends the raw public key. Server returns the app key, encrypted with the clientKey.

›

4 · EVERY CALL
ECDSA signature
SHA256withECDSA over transmission key + payload. **Now by the YubiKey.**

Private key: one use

It only signs requests. No ECDH, no decryption. So a hardware signer can replace it completely.

App key: decrypts data

It's an AES key used in software on every call. It can't live on the YubiKey, but it can be stored only wrapped by it.

▢ steps now performed on the YubiKey

DESIGN

Three layers, small seams

Application	Prompts for the PIN, creates a `YubiKeySigner`, passes it to `SecretsManagerOptions`. About 10 lines.
yubikey module new artifact	`YubiKeySigner` implements `RequestSigner` and `AppKeyProtector`. A minimal PIV client over `javax.smartcardio`: generate, sign, ECDH, metadata, certificates, device info. Java 8+, no dependencies besides core.
core SDK	Generic, knows nothing about YubiKeys: `RequestSigner` (sign, `keyLocation`, `onBound`), `AppKeyProtector` (wrap/unwrap), `initializeStorage(publicKey = …)`. Config fields: `publicKey`, `appKeyWrapped`, `privateKeyLocation`.
YubiKey PIV	PC/SC is built into macOS and Windows (`pcscd` on Linux). ECC P-256 required: YubiKey 4 / 5, firmware 5.3+ for automatic slots and app key wrapping.

ON THE YUBIKEY

Slot layout

9A Authentication	**9C** Signature	**9D** App key wrapping	**9E** Card auth	**82** Production App	**83** CI Pipeline	**84** Staging	**85** free
86 free	**87** free	**88** free	**89** free	**8A** free	**8B** free	**8C** free	**…95** free

One key per config

Each config gets its own P-256 key in the first free retired slot (82–95). A public key can be registered only once.

One shared wrapping key

9D, the slot PIV defines for key management, holds the ECDH key that wraps every config's app key.

Standard slots untouched

9A / 9C / 9E stay free for smart-card login, S/MIME and similar. Occupied slots are never overwritten.

Up to **20 KSM configs** per YubiKey. Slot names in YubiKey tools come from certificates the SDK writes.

FLOW

Adding a KSM device with a YubiKey

1 — Verify PIN
Before anything is written, so a wrong PIN can't leave a stray key.

›

2 — Generate key
P-256 in a free retired slot, then a sign/verify self-check.

›

3 — Wrapping key
Created in 9D on first use, marked by a certificate.

›

4 — initializeStorage
Stores the public key. No `privateKey`.

›

5 — Bind
First `getSecrets`: the app key arrives and is stored wrapped.

›

6 — Name the key
`onBound`: certificate named after the KSM app title.

```
// once, with a One-Time Access Token
YubiKeySigner(pin).use { yk ->
    initializeStorage(storage, token,
        publicKey = yk.generateKey())
    getSecrets(SecretsManagerOptions(storage,
        signer = yk, appKeyProtector = yk))
}
```

Steps 2, 3 and 6 need the management key

The factory default is used unless the app passes one. It's kept in memory only until binding names the certificate, then wiped.

▢ performed on the YubiKey

FLOW

Every run after that

1
Find the slot
`forPublicKey` matches the config's `publicKey` against slot metadata.

›

2
PIN once
Per connection. Re-verified automatically if another app resets the card.

›

3
Unwrap app key
ECDH on the YubiKey, once. Then held in memory until `close()`.

›

4
Sign requests
Each request is signed on the YubiKey. Retries and throttling re-sign transparently.

```
// every run
val publicKey = storage.getBytes(KEY_PUBLIC_KEY)!!
YubiKeySigner.forPublicKey(pin, publicKey).use { yk ->
    getSecrets(SecretsManagerOptions(storage,
        signer = yk, appKeyProtector = yk))
}
```

Wrong YubiKey plugged in?

A clear error instead of the server's "signature is invalid":

```
The key of this config is not on the connected YubiKey.
It is on: YubiKey 5C NFC FIPS, serial 34822070, PIV slot
84 (RETIRED3)
```

▢ performed on the YubiKey

UNATTENDED

Services, CI and cron: nobody to type the PIN

Permanently unattended: PIN-less keys

- `generateKey(pinPolicy = NEVER)`, then `YubiKeySigner()`
- Own wrapping key: **"...App Key Wrapping, no PIN"**
- No secret on the host: the factor is **having the YubiKey**

Temporarily unattended: PIN in the config

- `savePin(storage)` verifies, then saves `privateKeyPin`
- `removePin(storage)`: attended again
- The **last PIN retry** is always left for a person

```
// PIN asked for only when needed
val pin = PinProvider.fromStorage(storage) {
    readPin()
}
YubiKeySigner.forPublicKey(pin, publicKey)
```

Trade-off

A saved PIN is plain text: config plus YubiKey give full access. PIN policy is fixed at generation.

OPERATIONS

What you see in YubiKey tools



- PIV keys have no names, so tools show the subject of the slot's certificate.
- At binding the SDK writes a **self-signed certificate named after the KSM application** ("App5" here). It's signed by the slot's own key, on the YubiKey.
- Without it, every slot reads "Retired Key Management".
- Slot 9D is named **"Keeper Secrets Manager App Key Wrapping"**, the shared app key wrapping key.
- The same names appear in ykman and macOS Keychain Access.

SUMMARY

Nothing secret is left in the file

Software key

```
{
  "hostname": "keepersecurity.com",
  "clientId": "Gx3h…",
  "privateKey": "MIGHAgEAMBMG…",
  "appKey": "q9ZtR2…",
  "appOwnerPublicKey": "BGn0…"
}
```

YubiKey

```
{
  "hostname": "keepersecurity.com",
  "clientId": "Gx3h…",
  "publicKey": "BHmZ…",
  "privateKeyLocation":
    "YubiKey 5C NFC FIPS, serial 34822070,
     PIV slot 84 (RETIRED3)",
  "appKeyWrapped": "AQR5…",
  "appOwnerPublicKey": "BGn0…"
}
```

- The public key finds the slot, and the location tells a person which YubiKey to plug in.
- The model name is the one ykman shows, read from the YubiKey itself. The serial is printed on the key.
- Without the YubiKey, the config alone can't sign a request or decrypt a single record.